4DT906 Assignment 1: Parallel Computing
Odd-Even Sort and Pearson Correlation Coefficient (C/C++)

Samuel Berg, Jesper Wingren, Emil Ulvagården

March 6, 2026

Abstract

This report presents parallel implementations of two algorithms: the odd-even transposition sort and the Pearson correlation coefficient (PCC). Both algorithms are implemented using C++ `std::thread` for shared-memory parallelism. Correctness is validated against the provided sequential reference implementations. Performance results and speedup measurements are to be filled in after benchmarking.

Contents

1 Introduction **2**

2 Benchmarking Setup **2**

3 Odd-Even Transposition Sort **2**
3.1 Algorithm Description . . . 2
3.2 Sequential Implementation . . . 2
3.3 Parallel Implementation — `std::thread` (`oddevensort_par.cpp`) . . . 3
3.3.1 Parallelisation Strategy . . . 3
3.3.2 Synchronisation . . . 3
3.3.3 Data Decomposition . . . 3
3.4 Correctness Validation . . . 3
3.5 Performance Results — Odd-Even Sort . . . 3

4 Pearson Correlation Coefficient **3**
4.1 Algorithm Description . . . 3
4.2 Sequential Implementation . . . 4
4.3 Parallel Implementation — `std::thread` (`pcc_par.cpp`) . . . 4
4.3.1 Parallelisation Strategy . . . 4
4.3.2 Data Decomposition . . . 4
4.3.3 Synchronisation . . . 4
4.4 Correctness Validation . . . 4
4.5 Performance Results — Pearson Correlation Coefficient . . . 5

5 Discussion **5**
5.1 Scalability and Observed Behaviour . . . 5
5.2 Memory Behaviour . . . 5
5.3 Load Balancing . . . 5

6 Conclusion **5**

1 Introduction

Parallel computing enables large-scale computations to complete in less time by distributing work across multiple execution units. This assignment explores two algorithms that are amenable to parallelisation:

1. **Odd-Even Transposition Sort**: A comparison-based sorting algorithm with a regular, phase-alternating structure where independent pairs can be processed in parallel.
2. **Pearson Correlation Coefficient (PCC)**: A statistical measure of linear correlation between pairs of data rows. Computing all $\frac{m(m-1)}{2}$ pair-wise correlations for an $m \times n$ matrix is computationally intensive and highly parallelisable.

For each algorithm a correct sequential reference implementation is provided (`oddevensort.cpp` and `pcc_seq.cpp`). Parallel versions were developed on top of these references without modifying them.

2 Benchmarking Setup

- **Hardware**: [**TODO: fill in CPU model, core count, RAM**]
- **Operating System**: [**TODO: fill in OS and kernel version**]
- **Compiler**: g++ with `-O2` optimisation, C++17 standard
- **Thread counts tested**: [**TODO: e.g. 1, 2, 4, 8, . . .**]
- **Number of runs per configuration**: [**TODO: e.g. 3 runs, median reported**]

3 Odd-Even Transposition Sort

3.1 Algorithm Description

The odd-even transposition sort alternates between two phases until the array is sorted:

- **Even phase**: compare-and-swap all pairs $(v[0], v[1])$, $(v[2], v[3])$, ...
- **Odd phase**: compare-and-swap all pairs $(v[1], v[2])$, $(v[3], v[4])$, ...

After n full iterations (even + odd) the array is guaranteed to be sorted. Within each phase all active pairs are independent, making the algorithm naturally parallel.

3.2 Sequential Implementation

The reference implementation (`oddevensort.cpp`) performs n outer iterations and processes each active pair in the inner loop:

```
for (int i = 1; i <= s; i++) {
    for (int j = i % 2; j < s-1; j = j + 2) {
        if (numbers[j] > numbers[j + 1]) {
            std::swap(numbers[j], numbers[j + 1]);
        }
    }
}
```

Listing 1: Sequential Odd-Even Sort (core loop)

Time complexity: $O(n^2)$ in the worst case.

3.3 Parallel Implementation — `std::thread` (`oddevensort_par.cpp`)

3.3.1 Parallelisation Strategy

Within each phase the $\lfloor n/2 \rfloor$ active compare-swap pairs are independent. The pairs are divided into contiguous blocks, one block per thread, so threads operate on disjoint memory locations and no data races occur. A reusable spin-wait barrier synchronises all threads between phases, ensuring phase $k+1$ does not begin before phase k has finished.

3.3.2 Synchronisation

A custom `Barrier` class based on `std::atomic<int>` provides the inter-phase synchronisation. Each thread calls `arrive_and_wait()` at the end of its phase work; the last arriving thread resets the counter and increments a generation counter, waking the others.

3.3.3 Data Decomposition

Pairs are divided into contiguous blocks to maximise cache locality. Thread t handles pairs `[pair_start, pair_end)` where the block boundaries are computed to distribute the remainder evenly:

```
int per        = num_pairs / num_threads;
int rem        = num_pairs % num_threads;
int pair_start = thread_id * per + (thread_id < rem ? thread_id : rem);
int pair_end   = pair_start + per + (thread_id < rem ? 1 : 0);
```

Listing 2: Contiguous pair-block assignment

3.4 Correctness Validation

The parallel implementation is verified with `std::is_sorted` after execution. The default input size is $2^{19} = 524\,288$ elements; additional sizes were also tested.

3.5 Performance Results — Odd-Even Sort

Input size	Threads	Sequential (s)	Parallel (s)	Speedup
2^{19}	-	-	-	-
2^{19}	-	-	-	-
2^{19}	-	-	-	-

Table 1: Odd-Even Sort — `std::thread` implementation

4 Pearson Correlation Coefficient

4.1 Algorithm Description

The Pearson correlation coefficient between two vectors x and y of length n is:

$$r = \frac{\sum_{k=1}^{n}(x_k - \bar{x})(y_k - \bar{y})}{\sqrt{\sum_k (x_k - \bar{x})^2}\,\sqrt{\sum_k (y_k - \bar{y})^2}}$$

For a matrix with m rows and n columns, all $\frac{m(m-1)}{2}$ row-pair correlations are computed. The calculation proceeds in three steps:

1. Compute the row mean $\bar{x}_i$ for every row i.
2. Compute the mean-subtracted matrix $(x_{ik} - \bar{x}_i)$ and the row standard deviation σ_i.
3. Compute each pair-wise correlation $r_{ij} = \frac{\sum_k (x_{ik}-\bar{x}_i)(x_{jk}-\bar{x}_j)}{\sigma_i \sigma_j}$.

4.2 Sequential Implementation

The reference implementation (`pcc_seq.cpp`) computes all three steps in sequence. The correlation loop is:

```
for (sample1 = 0; sample1 < ROWS-1; sample1++) {
    for (sample2 = sample1+1; sample2 < ROWS; sample2++) {
        sum = 0.0;
        for (i = 0; i < COLS; i++) {
            sum += mm[sample1*COLS+i] * mm[sample2*COLS+i];
        }
        r = sum / (std[sample1] * std[sample2]);
        output[...] = r;
    }
}
```

Listing 3: Sequential PCC correlation loop

Time complexity: $O(m^2 n)$.

4.3 Parallel Implementation — `std::thread` (`pcc_par.cpp`)

4.3.1 Parallelisation Strategy

All three steps (mean, deviation/std, correlations) are parallelised. Steps 1 and 2 decompose the work by rows: each thread receives a contiguous slice of rows. Step 3 uses interleaved (round-robin) assignment of `sample1` rows across threads, which distributes the triangular workload evenly because rows with a higher index produce more pairs.

4.3.2 Data Decomposition

- **Steps 1 & 2 (linear)**: rows $[0, m)$ are split into contiguous blocks of size $\lceil m/T \rceil$ using a helper lambda.
- **Step 3 (triangular)**: thread t processes `sample1` rows t, $t+T$, $t+2T$, ... (stride T). Because each output index `sample1*ROWS + sample2 - offset` is written by exactly one thread, no synchronisation is required inside the correlation loop.

4.3.3 Synchronisation

No explicit synchronisation is needed within steps 1–3 because the work partitions are non-overlapping. Thread joining (`th.join()`) serves as the implicit barrier between steps.

4.4 Correctness Validation

The provided `verify` program compares the floating-point output files of the sequential and parallel implementations. The parallel version is validated for all tested matrix sizes.

Matrix size	Threads	Sequential (s)	Parallel (s)	Speedup
64×64	-	-	-	-
128×128	-	-	-	-
256×256	-	-	-	-
512×512	-	-	-	-
1024×1024	-	-	-	-
2048×2048	-	-	-	-
4096×4096	-	-	-	-

Table 2: PCC — `std::thread` implementation

4.5 Performance Results — Pearson Correlation Coefficient

5 Discussion

5.1 Scalability and Observed Behaviour

[TODO: fill in after benchmarking — describe speedup trends, crossover points, and any surprising results]

- **Odd-Even Sort**: The algorithm performs n phases with a barrier between each phase. Synchronisation overhead grows with thread count, potentially limiting scalability for small arrays.
- **PCC**: The correlation step dominates runtime for large matrices, making PCC well-suited for parallelisation. The implementation is expected to scale well for large matrices where computation time dominates overhead.
- **Thread overhead vs. Computation**: For small inputs, thread management overhead can exceed the computational savings, leading to reduced speedup. As problem size increases, the compute-to-overhead ratio improves.

5.2 Memory Behaviour

- **Odd-Even Sort**: Contiguous pair-block assignment improves spatial locality. Each thread accesses a contiguous region of the array, reducing cache-line contention between threads.
- **PCC**: Each thread reads the full mean-subtracted matrix rows it needs; rows are accessed sequentially, maintaining good cache behaviour.

5.3 Load Balancing

- **Odd-Even Sort**: The per-thread workload is perfectly balanced since each active phase has the same number of pairs and these are distributed evenly.
- **PCC**: The interleaved (round-robin) row assignment in step 3 distributes the triangular workload evenly across threads, avoiding the imbalance that would arise from a simple contiguous partition.

6 Conclusion

Both odd-even sort and the Pearson correlation coefficient were parallelised successfully using C++ `std::thread` for shared-memory execution. Correctness was validated against the provided sequential reference programs.

Performance results and speedup values will be added after benchmarking on the target hardware. The implementations are expected to achieve positive speedup ($> 2\times$) for non-trivial input sizes, with efficiency improving as problem size grows and the compute-to-overhead ratio increases.

References

- Quinn, M. J. *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill, 2004.
- Course material: `pa1_26.pdf` — Programming Assignment 1 instructions.